CS50x 2026 - Lecture 3 Notes - Algorithms

By Wisdom

Phonebook.c

In C, we can create a simple phonebook program by combining **structures, arrays, and loops**. First, we define a `struct` called `Contact` that contains two pieces of information for each contact: a name and a phone number. Using a struct allows us to keep all related information together as a single object. Then we create an array of these structs to store multiple contacts. Since C arrays need a fixed size, we define a maximum number of contacts using a constant. To track how many contacts are currently in the phonebook, we use a counter variable. This counter is crucial because it tells us which parts of the array are actually used.

To interact with the user, we create a **menu-driven loop** using a `do while` statement. This loop repeatedly shows the user options to add, search, delete, display contacts, or exit the program. Inside the loop, we use a `switch` statement to call the appropriate function based on the user's choice. This keeps the program organized and easy to follow.

The **add contact function** prompts the user for a name and phone number, stores the information in the next available slot in the array, and increments the counter. We use a specific `scanf` format that reads input including spaces, which ensures names like "Okechukwu Wisdom" are captured correctly. We also check if the array is full before adding a contact to prevent memory issues.

The **search function** loops through all existing contacts and compares each contact's name with the name provided by the user. If a match is found, the program prints the contact's information and stops searching. If no match is found, the program informs the user. We use a counter in the loop to keep track of which contact we are checking and `strcmp` to compare strings correctly.

The **delete function** is slightly more complex. First, we loop through the array to find the contact that matches the user's input. Once we find it, we need to remove it from the array. Since arrays cannot shrink automatically, we shift all contacts after the deleted contact one position to the left. This fills the gap left by the deleted contact and keeps the array compact. After shifting, we decrement the counter to reflect that one contact has been removed. If no contact matches the input, the program informs the user.

The **display function** loops through all the contacts in the array and prints their name and phone number. We use `i + 1` to make the numbering more user-friendly. If there are no contacts, the program simply prints that the phonebook is empty.

Overall, this program demonstrates how to combine structs, arrays, counters, loops, and conditional statements to manage a small dataset. Using these basic tools, we can build an interactive program that allows users to add, search, delete, and display information efficiently.

```c
// Function to search for a contact
void searchContact() {
    char name[50];
    printf("Enter name to search: ");
    scanf(" %[^\n]", name);
    for (int i = 0; i < contactCount; i++) {
        if (strcmp(phonebook[i].name, name) == 0) {
            printf("Found: %s - %s\n", phonebook[i].name, phonebook[i].phone);
            return;
        }
    }
    printf("Contact not found.\n");
}

// Function to delete a contact
void deleteContact() {
    char name[50];
    printf("Enter name to delete: ");
    scanf(" %[^\n]", name);
    for (int i = 0; i < contactCount; i++) {
        if (strcmp(phonebook[i].name, name) == 0) {
            for (int j = i; j < contactCount - 1; j++) {
                phonebook[j] = phonebook[j + 1]; // Shift left
            }
            contactCount--;
            printf("Contact deleted successfully!\n");
            return;
        }
    }
    printf("Contact not found.\n");
}
```

Searching

Linear Search and Binary Search

```c
int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Linear search is the simplest way to find an element in an array. The program starts at the **first element** and checks if it matches the value we are looking for. If it does, the search **stops immediately** and returns the index of that element. If not, it moves to the **next element**, and the process repeats. This continues until either the element is found or the program reaches the **end of the array**. If the element is not in the array, the function returns `-1` to indicate it wasn't found. Linear search works on **any array**, whether sorted or unsorted, because it examines each element one by one. While it is easy to implement and understand, it can be slow for very large arrays because it may need to check **every single element**.

Binary search is a faster method, but requires the array to be **sorted** first. Instead of checking every element, it starts by looking at the **middle element**. If the middle element is equal to the target, the search is done. If the target is **smaller** than the middle element, the program ignores the right half and continues searching in the **left half**. If the target is **larger**, it ignores the left half and searches in the **right half**. This process of dividing the array in half repeats until the target is found or there are no more elements to check. Each step **eliminates half the array**, making binary search **much faster** than linear search for large arrays. Its efficiency comes from this "divide and conquer" approach, which quickly narrows down the possible positions of the target element.

```c
int binarySearch(int arr[], int size, int target) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

Running Time

Running time describes how the time an algorithm takes grows as the size of the input increases. Instead of measuring the exact number of seconds, we describe this growth using **asymptotic notation**, such as **Big-O, Big-Ω (Omega), and Big-Θ (Theta)**. **Big-O** represents the **upper bound**, meaning the worst-case running time an algorithm could take. For example, in **linear search**, the algorithm may need to check every element in the array before finding the target or determining it is not present, so the worst-case running time is **O(n)**. **Big-Ω (Omega)** represents the **lower bound**, which describes the best-case scenario. In linear search, if the target element is the first item in the array, the algorithm finds it immediately, so the best-case running time is **Ω(1)**.

Big-Θ (Theta) represents the **tight bound**, meaning the running time grows consistently between the upper and lower bounds. Many algorithms are also described using expressions like **n, log n, or n^2**, which show how quickly the work increases as the input grows. For example, **binary search** runs in **O(log n)** time because it repeatedly divides the search space in half, making it much faster for large datasets compared to linear search. Understanding these growth rates helps us evaluate how efficient an algorithm is and compare different algorithms when solving the same problem.

Sorting

Sorting in C is the process of arranging numbers in a particular order, usually from smallest to largest. One simple type is **bubble sort**, where you repeatedly compare two neighboring numbers and swap them if they are in the wrong order. For example, in `[5, 2, 4]`, you compare 5 and 2 and swap them → `[2, 5, 4]`, then compare 5 and 4 and swap again → `[2, 4, 5]`. You keep repeating this until everything is sorted.

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Another type is **selection sort**, where you repeatedly find the smallest number and move it to the front. For example, in `[5, 2, 4]`, you first find the smallest number (2) and swap it with 5 → `[2, 5, 4]`, then find the next smallest (4) and swap it with 5 → `[2, 4, 5]`. This continues until the whole list is sorted.

```
void selectionSort(int arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for(int j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

Insertion sort works like arranging cards in your hand—you take one number at a time and insert it into the correct position in the already sorted part. For example, in `[5, 2, 4]`, you start with 5, then take 2 and insert it before 5 → `[2, 5, 4]`, then take 4 and place it between 2 and 5 → `[2, 4, 5]`.

```
void insertionSort(int arr[], int n) {
    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Finally, **quick sort** is a faster method that picks a “pivot” number and splits the list into smaller and larger numbers around it. For example, in `[5, 2, 4]`, if 4 is the pivot, you put smaller numbers like 2 on one side and larger ones like 5 on the other → `[2, 4, 5]`. Then you repeat this process on each side until everything is sorted.

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] < pivot) {
```

```
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Recursion

Recursion is a programming technique where a function calls itself to solve smaller versions of a problem until it reaches a stopping point called the base case. The idea is to keep reducing the problem until it becomes simple enough to solve directly. For example, factorial: `5! = 5 × 4 × 3 × 2 × 1`. Recursively, this becomes `factorial(5) = 5 × factorial(4)`, `factorial(4) = 4 × factorial(3)` … until `factorial(1) = 1` (base case), then it resolves back up to give `120`.

```
#include <stdio.h>
int factorial(int n) {
    if(n == 1) return 1;
    return n * factorial(n - 1);
}
```

Merge sort is a fast sorting algorithm that uses recursion to divide an array into smaller parts, sort them, and then merge them back together. The array is repeatedly split in half until each part has one element, then those parts are merged in order. For example, `[5, 2, 4]` is split into `[5]` and `[2, 4]`, then `[2, 4]` is split into `[2]` and `[4]`. When merging, `[2]` and `[4]` become `[2, 4]`, and then `[5]` merges with `[2, 4]` to form `[2, 4, 5]`.

```
void mergeSort(int arr[], int left, int right) {
    if(left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        // merging logic happens here
    }
}
```

Sorting race is the idea of comparing different sorting algorithms to see which one performs faster on the same data. It's not a new algorithm but a way to measure efficiency. For example, if you sort `[5, 2, 4, 1, 3]` using bubble sort and merge sort, bubble sort will take many comparisons and swaps, while merge sort splits and merges efficiently, finishing much faster for larger arrays. Programmers often use timing functions to measure this difference.

```
#include <time.h>
clock_t start = clock();
// call sorting function here
clock_t end = clock();
double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
```